ENSAM

ÉCOLE NATIONALE SUPÉRIEURE D'ARTS ET MÉTIERS
UNIVERSITÉ HASSAN II DE CASABLANCA

Rapport du Projet :

Maison Connectée

MSEI-CI1

Réalisée par:

Sara Manouni
Mehdi Fertoussi
Islam Bouzid

Encadré par: M. MOUMNI

www.realygreeksite.com

Remerciements

Nous tenons à exprimer nos remerciements les plus sincères à notre professeur, M. MOUMNI, pour son accompagnement constant, sa disponibilité et la qualité exceptionnelle de son enseignement.
Au cours des différentes séances du module Programmation Embarquée, nous avons acquis un ensemble riche et varié de compétences théoriques et pratiques :

- la maîtrise de l'environnement Thonny pour programmer des microcontrôleurs ;
- la compréhension du fonctionnement interne de l'ESP32 et des protocoles utilisés ;
- la gestion des entrées-sorties, des capteurs, des librairies et du débogage
- l'écriture d'un code structuré, optimisé et fiable ;
- et surtout le développement d'une méthode de travail rigoureuse, autonome et collaborative.

Nous remercions chaleureusement notre professeur pour sa pédagogie, sa patience et sa volonté de nous transmettre son expertise. Grâce à ses explications claires et à la progression qu'il a instaurée séance après séance, nous avons pu avancer avec assurance et dépasser plusieurs difficultés techniques.
Notre collaboration nous a permis de partager les tâches, d'apprendre les uns des autres et de produire un projet solide et complet.

Sommaire

INTRODUCTION ---------- 04
OBJECTIFS ---------- 05
MATÉRIEL UTILISÉ ---------- 06
LOGICIEL ET ENVIRONNEMENT ---------- 08

ARCHITECTURE DU SYSTÈME ---------- 09

MISE EN PLACE DE LA COMMUNICATION WI-FI ET FIREBASE ---------- 10

DIFFICULTÉS RENCONTRÉES ---------- 12
CONCLUSION ---------- 12

Introduction

Ce rapport s'inscrit dans le cadre du module de Programmation Embarquée, un enseignement essentiel qui nous a permis de découvrir, comprendre et maîtriser les bases du développement sur microcontrôleur. Au fil des séances, nous avons été amenés à manipuler l'environnement Thonny, un outil simple mais puissant, destiné à la programmation en Python et MicroPython.

Grâce à cet environnement, nous avons pu explorer en profondeur le fonctionnement du microcontrôleur ESP32, expérimenter différents programmes, tester des capteurs et développer des prototypes fonctionnels.

L'objectif général de ce projet est de renforcer notre autonomie et nos compétences pratiques en développant une approche complète allant de la compréhension théorique à l'implémentation réelle. Pour cela, nous avons étudié la structure interne de l'ESP32, son architecture matérielle, ses broches GPIO, ses modes d'alimentation, ainsi que les différents protocoles de communication qu'il supporte. Cette approche nous a permis de mieux comprendre comment un système embarqué interagit avec son environnement.

2. Objectifs

2.1 Surveillance environnementale

- Mesurer la température et l'humidité avec le capteur DHT22.
- Détecter la pluie via un capteur analogique de précipitations.
- Détecter la présence de gaz inflammable avec le MQ-2.

2.2 Affichage et alertes locales

- Afficher toutes les valeurs sur un LCD 16×2 avec mise à jour périodique.
- Déclencher des alertes visuelles et sonores (LED + buzzer) si les valeurs dépassent des seuils prédéfinis.

2.3 Communication et stockage des données

- Envoyer les données vers Firebase Realtime Database, pour stockage et consultation à distance.
- Permettre la modification à distance des seuils via Firebase, sans besoin de reprogrammer l'ESP32.

2.4 Test et débogage

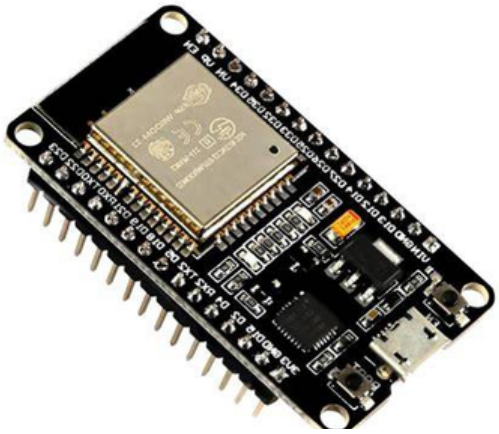
- Utiliser Thonny pour tester chaque capteur indépendamment, vérifier la communication Wi-Fi et s'assurer de la validité des données envoyées à Firebase.
- Détecter et corriger les erreurs dans les scripts MicroPython, optimisant ainsi la fiabilité du système.

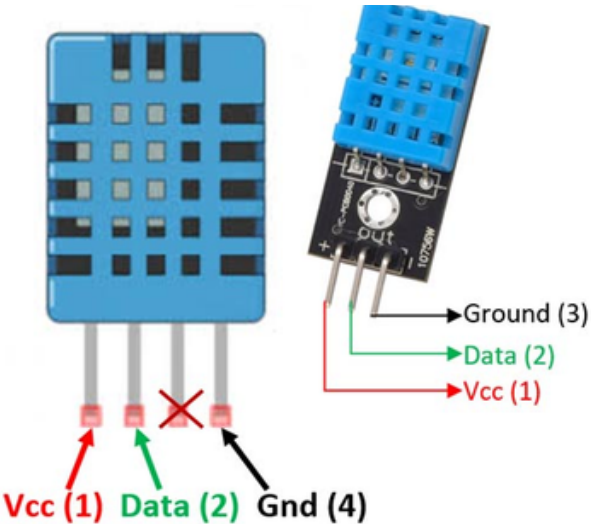
3. Matériel utilisé

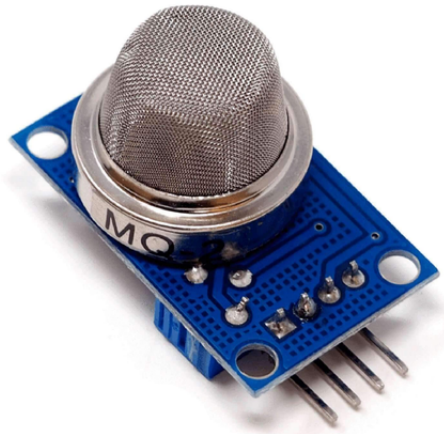
Composant	Rôle
ESP32	Microcontrôleur avec Wi-Fi intégré
DHT22	Capteur température et humidité
Capteur de pluie analogique	Détection de précipitations
Capteur de gaz MQ-2	Détection de gaz inflammables
LCD 16×2 I2C	Affichage des valeurs locales
LED + buzzer	Indication visuelle et sonore des alertes
Breadboard, câbles et alimentation 5V ;	Connexion alimentation du circuit

Explications techniques :

- **ESP32 :** microcontrôleur puissant, avec Wi-Fi intégré, adapté à la collecte et à l'envoi de données en temps réel.
- **DHT22** : capteur numérique offrant une précision correcte pour la température (±0,5 °C) et l'humidité (±2 %).
- **MQ-2 :** capteur analogique capable de détecter des concentrations de gaz inflammables comme le propane, le méthane et le butane.
- **LCD I2C :** simplifie le câblage grâce à un bus I2C unique pour SDA et SCL.







4. Logiciel et environnement

4.1 Thonny

Thonny est un IDE Python simple et efficace :

- Permet l'édition et l'exécution directe des scripts MicroPython sur ESP32.
- Possède un débogueur intégré pour surveiller les variables et détecter les erreurs.
- Interface claire : console intégrée, affichage des erreurs, gestion des modules.

4.2 MicroPython

- Version de Python adaptée aux microcontrôleurs.
- Librairies utilisées : machine, time, urequests, dht, I2C LCD.

4.3 Firebase Realtime Database

- Stockage en ligne des données pour consultation et suivi.
- Permet la lecture et modification des seuils depuis n'importe quel appareil connecté.
- Communication via requêtes HTTP (PUT/GET).

5.Architecture du système

1. **Acquisition des données**
   - Lecture des capteurs toutes les 60 secondes.
   - Vérification des valeurs lues et conversion si nécessaire.
2. **Affichage local**
   - LCD mis à jour en temps réel.
   - Formatage des données pour lisibilité (ex : T:25°C G:300).
3. **Alertes locales**
   - LED allumée et buzzer activé si seuil dépassé.
   - Sécurité immédiate pour les utilisateur**s.**
4. **Transmission des données vers Firebase**
   - Requêtes HTTP PUT envoyées toutes les minutes.
   - Données organisées en JSON : température, humidité, gaz, pluie et état d'alerte.
5. **Paramétrage à distance**
   - Lecture dynamique des seuils depuis Firebase.
   - Adaptation automatique de l'ESP32 sans reprogrammation.

•

6. Mise en place de la communication Wi-Fi et Firebase

Dans ce projet, une étape essentielle consistait à permettre à l'ESP32 de communiquer en temps réel avec une base de données en ligne. Pour cela, nous avons utilisé Firebase Realtime Database comme plateforme cloud, ainsi que le module network de MicroPython pour établir une connexion Wi-Fi. Toute la programmation a été réalisée dans l'environnement Thonny, ce qui nous a permis d'envoyer et de tester le code directement sur la carte ESP32.

6.1 Connexion Wi-Fi de l'ESP32

La première partie du programme consiste à connecter la carte au réseau Wi-Fi local. Nous utilisons pour cela le module network, qui active l'interface WLAN en mode station (STA_IF). Une boucle d'attente permet de vérifier la connexion jusqu'à ce que l'ESP32 obtienne une adresse IP. L'objectif est de garantir que la carte puisse accéder à Internet avant d'envoyer ou recevoir des données depuis Firebase. Cette connexion constitue la base de toutes les fonctionnalités "maison connectée".

6.2 Communication avec Firebase (lecture et écriture)

Une fois connectée, la carte utilise le module urequests pour effectuer des requêtes HTTP.

Deux fonctions essentielles ont été développées :

- **send_to_firebase()** : permet d'envoyer une valeur à un chemin précis dans la de données Firebase via une requête HTTP PUT.
- **read_from_firebase()** : permet de lire en temps réel l'état ou les données stockées dans Firebase.

Ces deux fonctions offrent une grande flexibilité : elles permettent de transmettre n'importe quelle donnée provenant des capteurs, mais aussi de recevoir des commandes envoyées par l'utilisateur via l'application ou l'interface Web.

6.3 Système d'alerte en cas de danger

Nous avons également intégré une fonction très importante, destinée à augmenter la sécurité de la maison connectée :

→ **send_danger_alert()**

Cette fonction s'exécute lorsqu'un seuil de gaz anormal est détecté ou lorsqu'un risque est présent (par exemple présence de pluie). Elle envoie automatiquement vers Firebase un message structuré contenant :

- un avertissement,
- la valeur du capteur de gaz,
- l'état météorologique (pluie ou sec),
- un conseil de sécurité.

Cette alerte peut ensuite être affichée sur l'interface mobile ou déclencher d'autres actions automatiques.

6. Difficultés Rencontrées

Quelques obstacles techniques ont été rencontrés :

- configuration du port USB ;
- erreurs de connexion ou de reconnaissance de la carte ;
- gestion du firmware et des versions incompatibles ;
- erreurs d'exécution liées à la syntaxe MicroPython.

Chaque difficulté a été résolue grâce à l'analyse, la recherche et l'aide du professeur.

11. Conclusion

Ce projet a été très enrichissant. Grâce à Thonny, nous avons pu coder, tester et déboguer le programme facilement. L'intégration de Firebase a permis de créer un système de surveillance à distance, tandis que les alertes locales ont assuré la sécurité immédiate.

Nous remercions encore notre professeur pour nous avoir donné cette opportunité, qui nous a permis d'acquérir des compétences pratiques précieuses, de la conception électronique à la programmation et à la gestion de données cloud.

Ce projet constitue une base solide pour des extensions futures, comme la domotique complète, les notifications avancées et l'optimisation de l'autonomie énergétique.